

1.

①. $N * (\log_2 N)^2 + N^2 * (\log_2 N)$

Expression = sum $\Rightarrow$ term with the fastest growth dominates:

$N(\log_2 N)^2 \ll N^2 \log_2 N < N^3 \log_2 N < N^5 \dots$

Dominant term: $N^2 * \log_2 N \Rightarrow \Theta(N^2 \log_2 N) \Rightarrow \boxed{c)} \rightarrow$ exact same complexity

$O(N^3 \log_2 N) \boxed{d)} \Rightarrow$ upper bound, $N^2 \log_2 N < N^3 \log_2 N$.

$\Omega(N(\log_2 N)^2) \boxed{a)} \rightarrow$ lower bound, $N(\log_2 N)^2 < N^2 \log_2 N$

② $\boxed{c)}$ remove an element from a position i in the context of a set implemented on a BA because a SET has NO POSITIONS and when we remove we remove by VALUE, not by POSITION

a) $\rightarrow$ a set must allow insertion, so add on a SET is valid.

b) $\rightarrow$ any container needs an iterator, so iterator on a SET is valid.

d) $\rightarrow$ the number of the elements is valid to be known.

e) $\rightarrow$ searching for an element in a SET is valid.

SET/BAG $\Rightarrow$ no positions assigned to elem.

*! get(position) would also have been invalid.

③ Separate chaining $\Rightarrow$ each slot = LINKED LIST of elements that hash there.

ht with 50 positions

collision res. with separate chaining

nr. of elements that can be inserted in the HT?

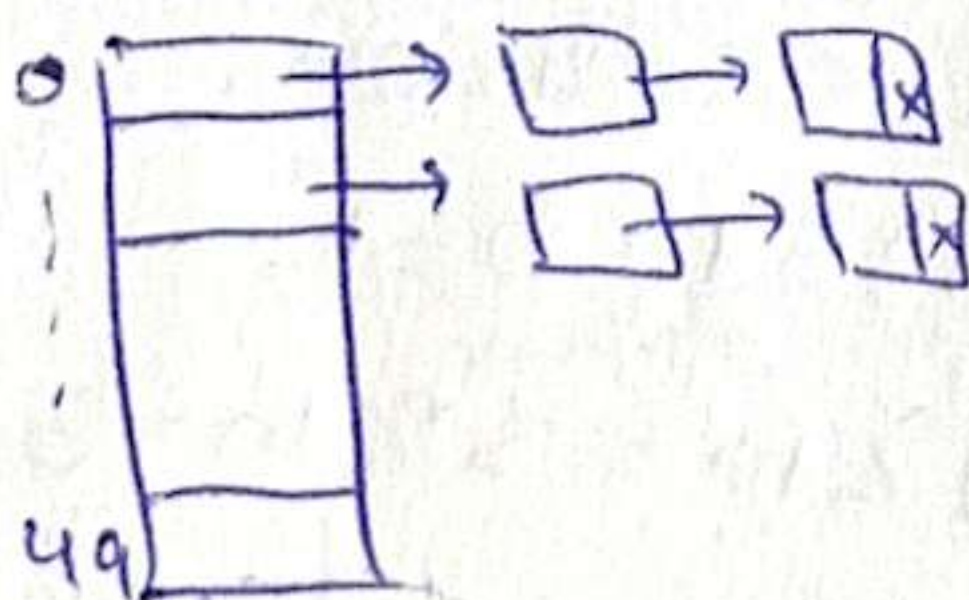
$\boxed{a)}$ 25 $\rightarrow$ possible, but might leave some spaces empty.

$\boxed{b)}$ 50 $\rightarrow$ one element/slot

$\boxed{c)}$ 49 $\rightarrow$ 49 occupied positions.

$\boxed{d)}$ 100 $\rightarrow$ 2 elements/slot (100 is possible because in separate chaining each table position contains a linked list, therefore multiple elements can be stored in the same pos.)

HT with 50 positions



④ 4 5 1 * - 6 3 + 2 * -

POSTFIX $\Rightarrow$ STACK.

operand $\Rightarrow$ push

operator $\Rightarrow$ pop last two operands from the stack

compute $\Rightarrow$ push result back to the stack

①

4 5 1 * - 6 3 + 2 * -

Token	Stack
4	4
5	4, 5
1	4, 5, 1
*	$\Rightarrow 5 * 1 = 5$
-	4, 5
6	-1 $\Rightarrow 4 - 5 = -1$
3	-1, 6
+	-1, 6, 3 $\Rightarrow 6 + 3 = 9$
2	-1, 9
*	-1, 9, 2 $\Rightarrow 9 * 2 = 18$
-	-1, 18
	$\Rightarrow -1 - 18 = -19$

②. HT with $m=10$ positions
open addressing with double hashing
division method

for the second hash function $\rightarrow h_2(k) = 1 + (k \bmod 9)$

Insert: 19, 37, 2, 99, 43, 72, 54, 17.

Final HT? Pos. for each elem.?

Open addressing with double hashing:
 $h(k, i) = (h'(k) + i \cdot h''(k)) \bmod m$
 $h'(k) = k \bmod m$ (div. method)
 $h''(k) = 1 + (k \bmod 9)$

Lec 10

• 19: $h'(19) = 19 \% 10 = 9 \checkmark$

• 37: $h'(37) = 37 \% 10 = 7 \checkmark$

• 2: $h'(2) = 2 \% 10 = 2 \checkmark$

• 99: $h'(99) = 99 \% 10 = 9 \Rightarrow$ collision

try pos. 1 $h''(99) = 1 + (99 \% 9) = 1 + 0 = 1$

$h(9, 1) = (9 + 1 \cdot 1) \% 10 = 10 \% 10 = 0 \checkmark$

• 43: $h'(43) = 43 \% 10 = 3 \checkmark$

• 72: $h'(72) = 72 \% 10 = 2 \Rightarrow$ collision.

try pos. 1 $h''(72) = 1 + (72 \% 9) = 1 + 0 = 1$

try pos. 2 $h(72, 1) = (2 + 1 \cdot 1) \% 10 = 3 \% 10 = 3 \times$

$h(72, 2) = (2 + 2 \cdot 1) \% 10 = 4 \% 10 = 4 \checkmark$

• 54: $h'(54) = 54 \% 10 = 4 \Rightarrow$ collision.

$h''(54) = 1 + (54 \% 9) = 1 + 0 = 1$

try pos. 1 $h(54, 1) = (4 + 1 \cdot 1) \% 10 = 5 \% 10 = 5 \checkmark$

0	99	99
1		
2	2	2
3	43	43
4	72	72
5	54	54
6	17	17
7	37	37
8		
9	19	19

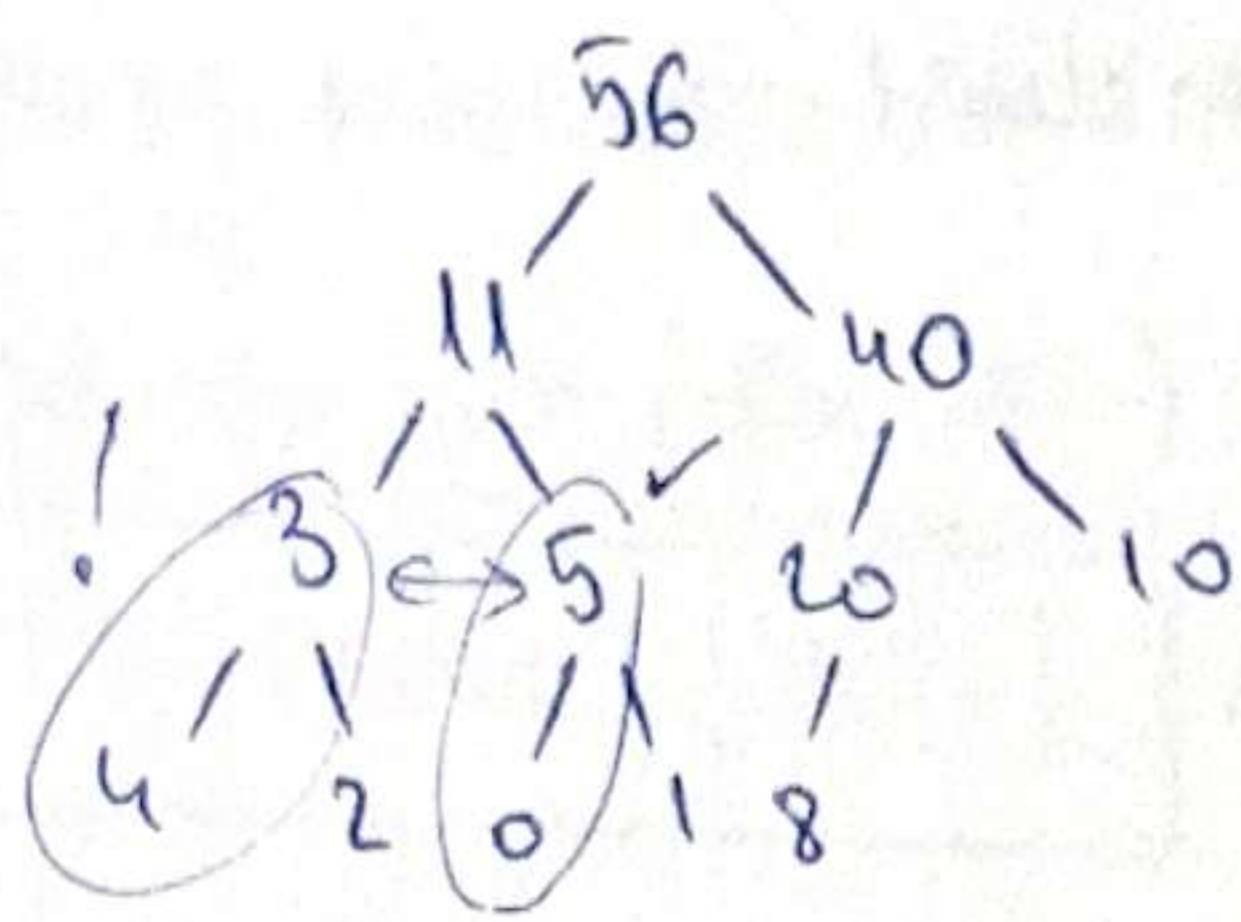
• 17: $h'(17) = 17 \% 10 = 7 \Rightarrow$ coll

$h''(17) = 1 + (17 \% 9) = 1 + 8 = 9$

$h(17, 1) = (7 + 9) \% 10 = 6 \checkmark$

②

a) Is $[56, 11, 40, 3, 5, 20, 10, 4, 2, 0, 1, 8]$ a heap?



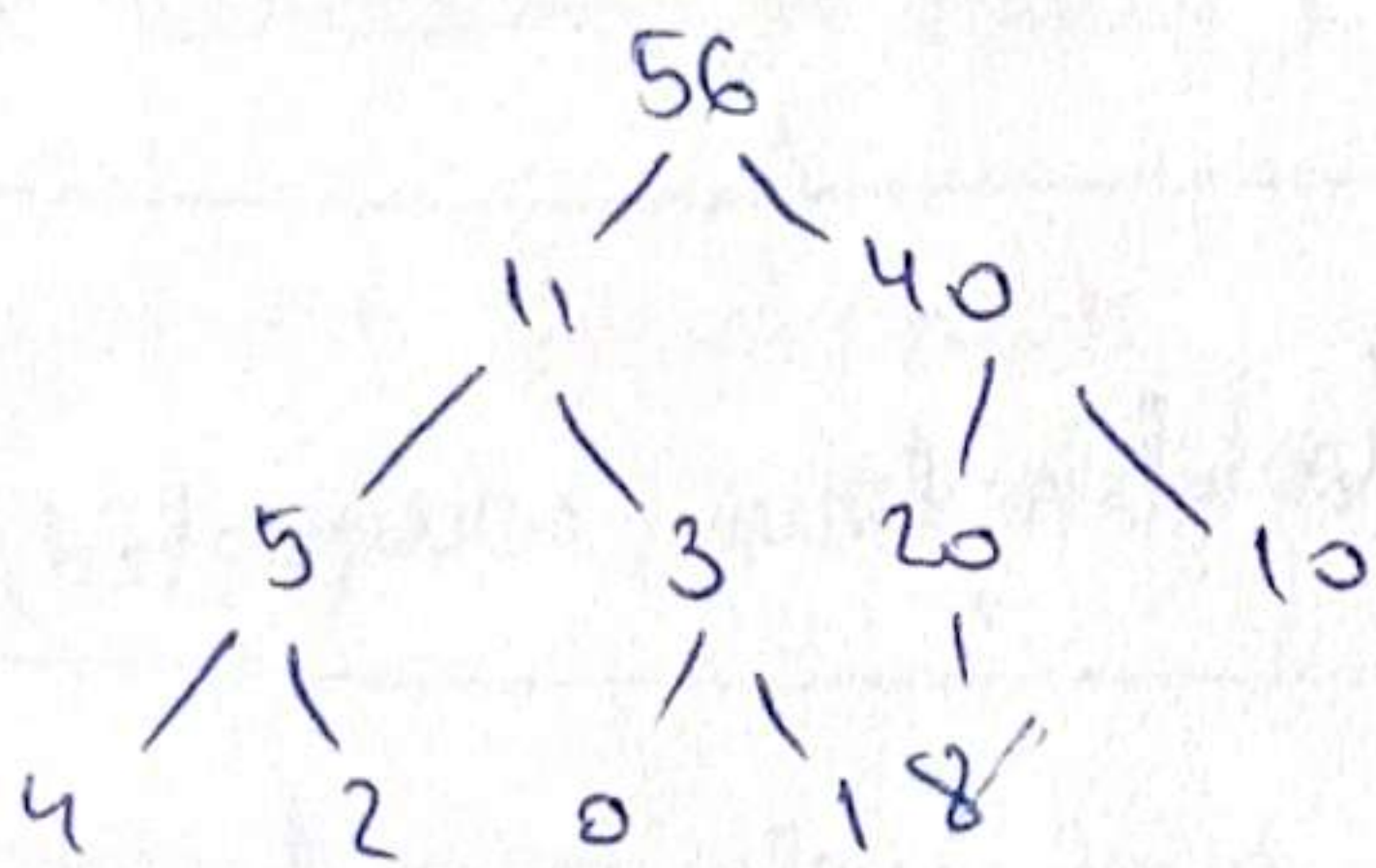
Not a heap because although it has structure property (no gaps), it does not have the property (max) because $3 < 4$, so it does not respect the max heap property.

A heap has structure + property (no gaps) (min/max)

b) Make it a heap by swapping two CONSECUTIVE elems. in the array.

$\Rightarrow 3 \leftrightarrow 5$.

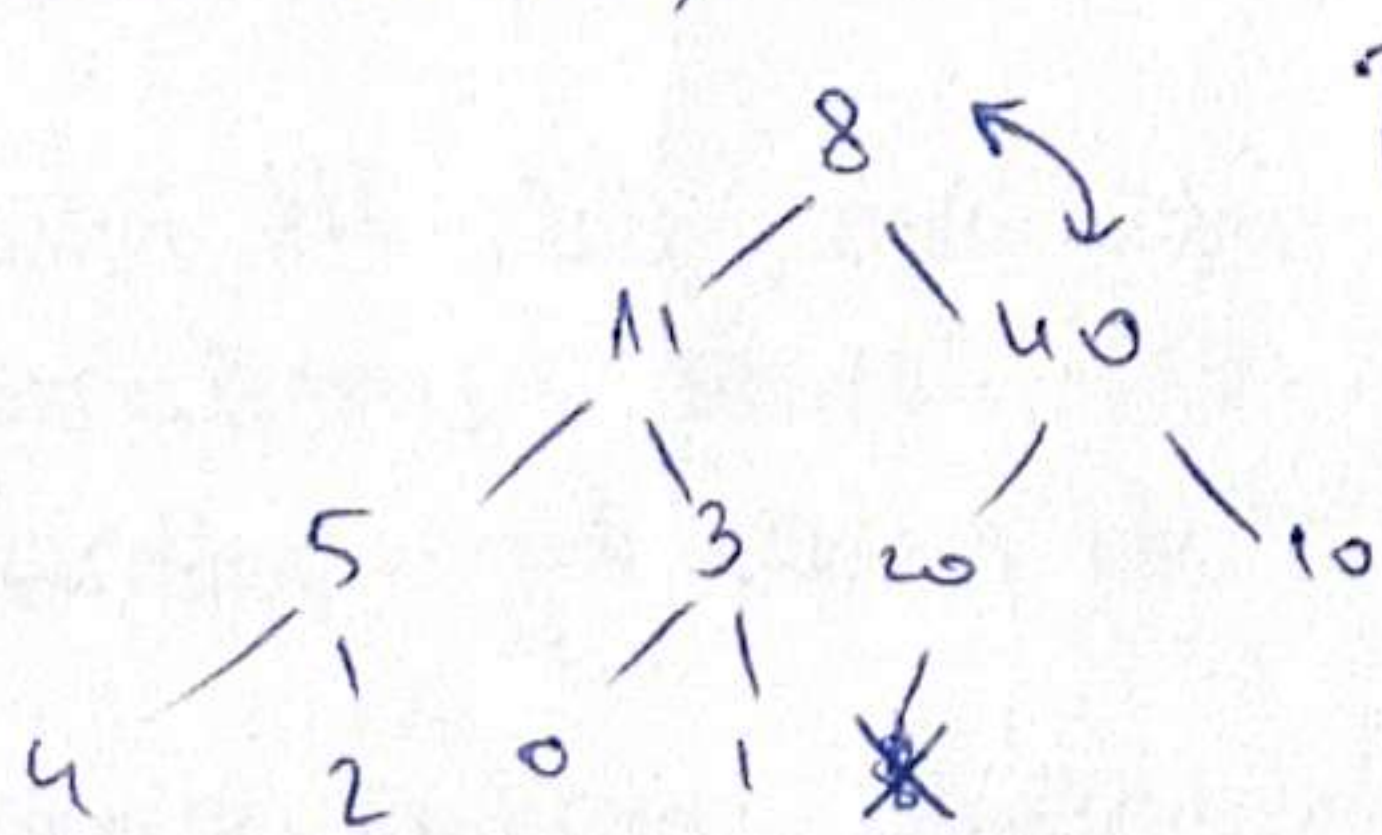
$\Rightarrow [56, 11, 40, 5, 3, 20, 10, 4, 2, 0, 1, 8]$



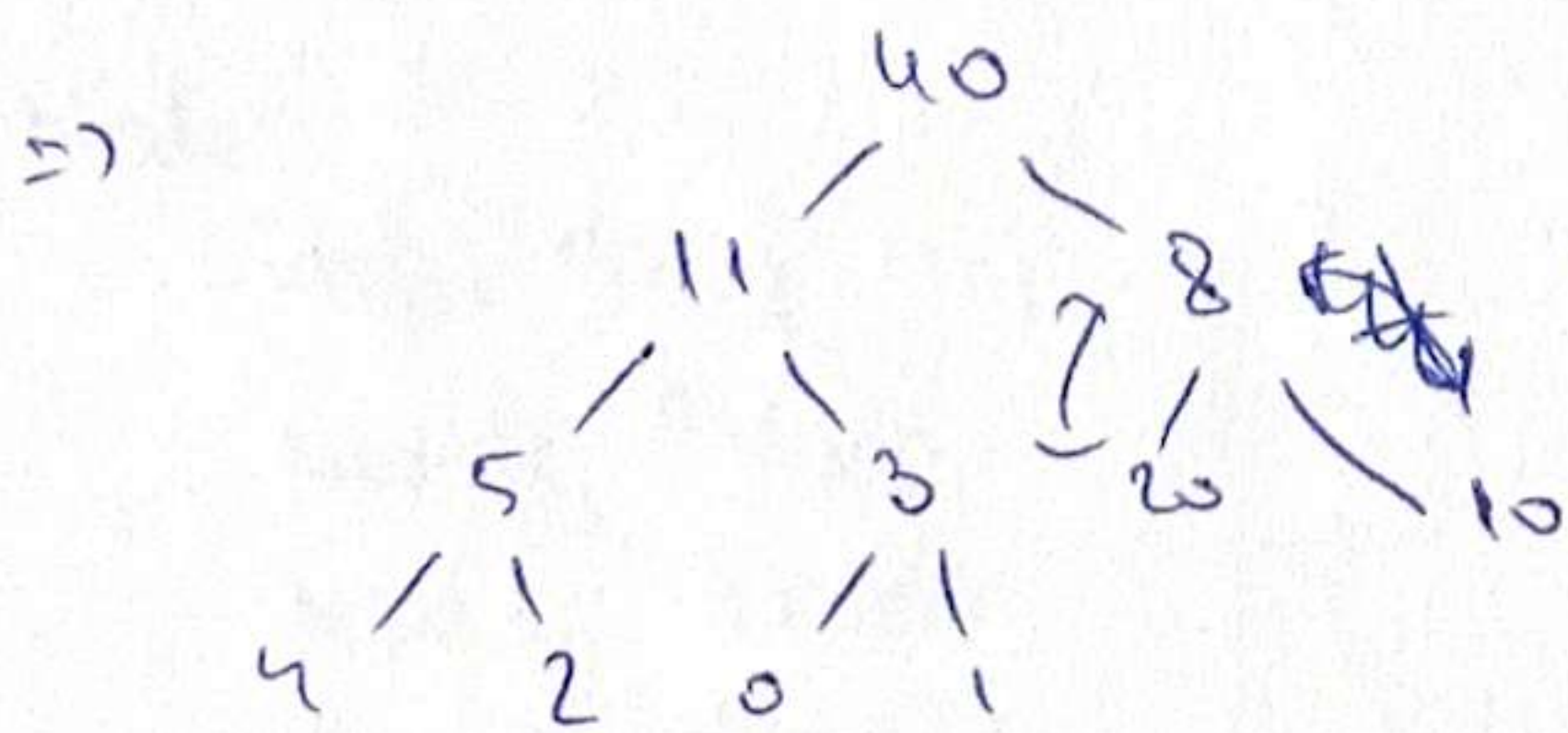
c) remove 3 elements (ROOT only)

$\rightarrow$ make last elem. to be the root
 $\rightarrow$ bubble-down (max/min of children $\rightarrow$ swap)

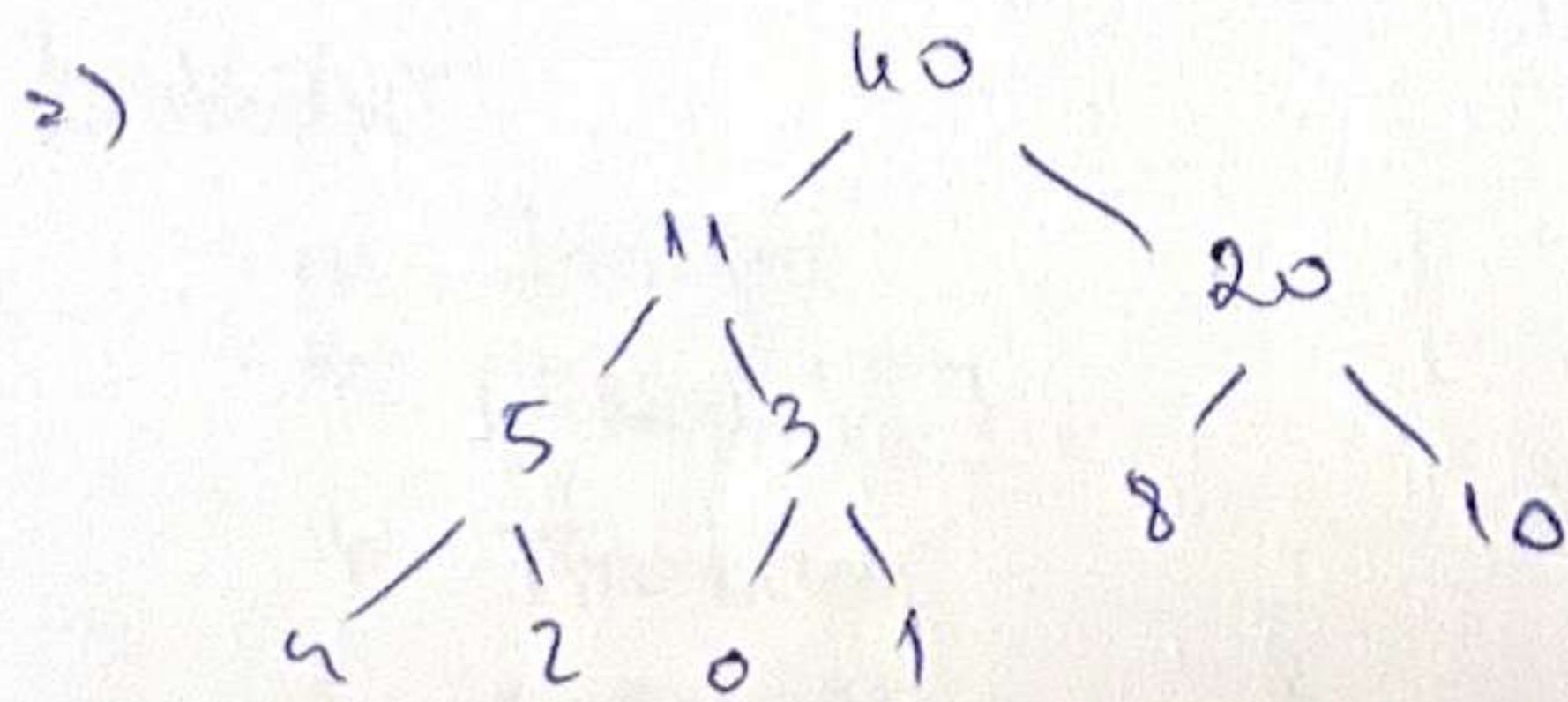
• remove 56 (root).



$\Rightarrow$ max child = 40 $\Rightarrow 8 \leftrightarrow 40$.

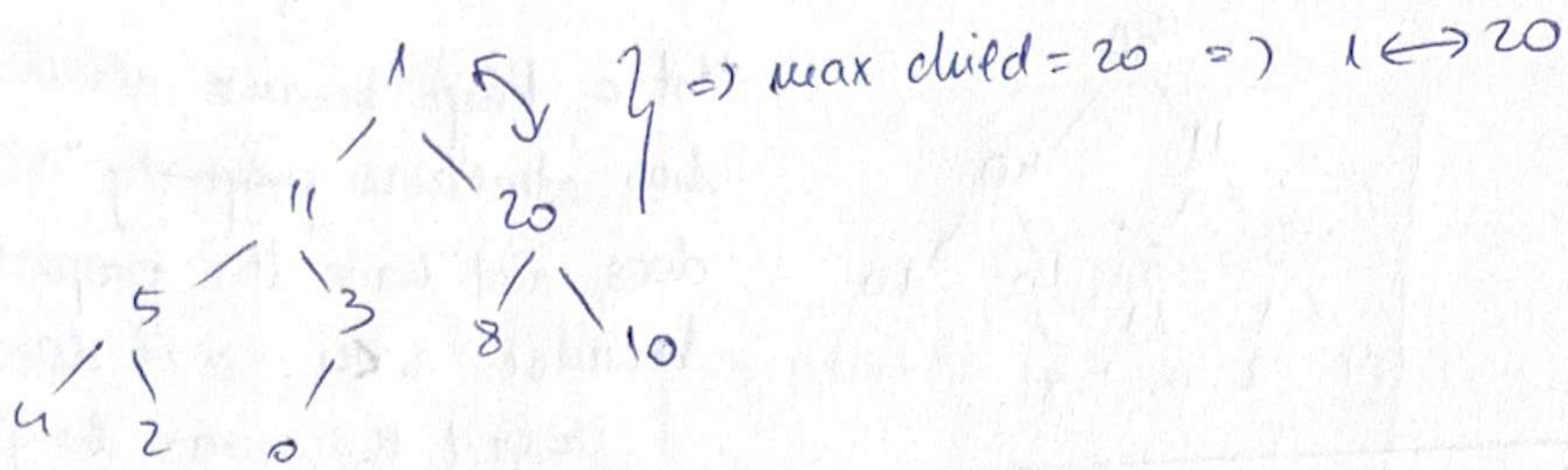


$\Rightarrow$ max child = 20 $\Rightarrow 8 \leftrightarrow 20$

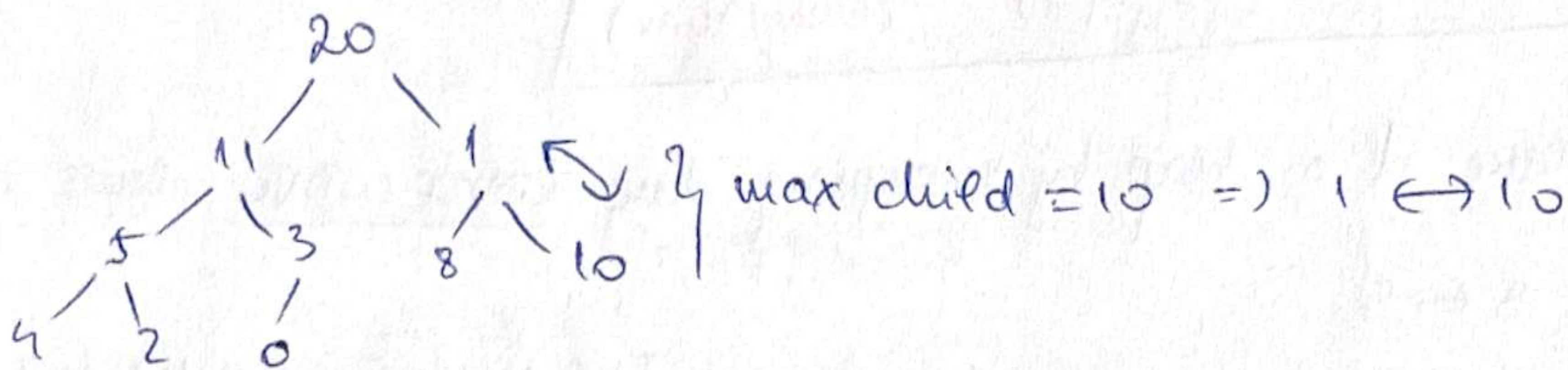


bubble-down completed.

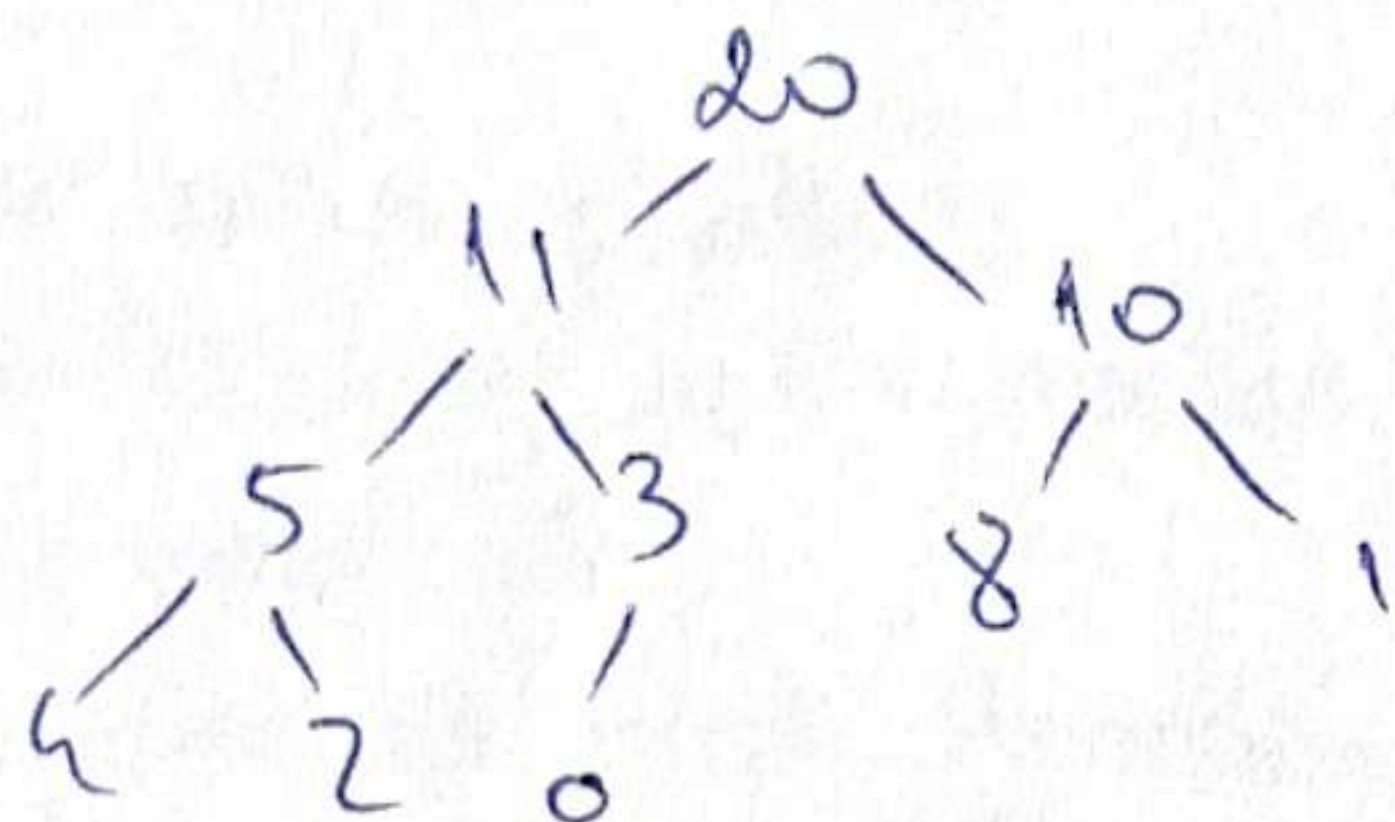
• Remove 40 (root)



=>

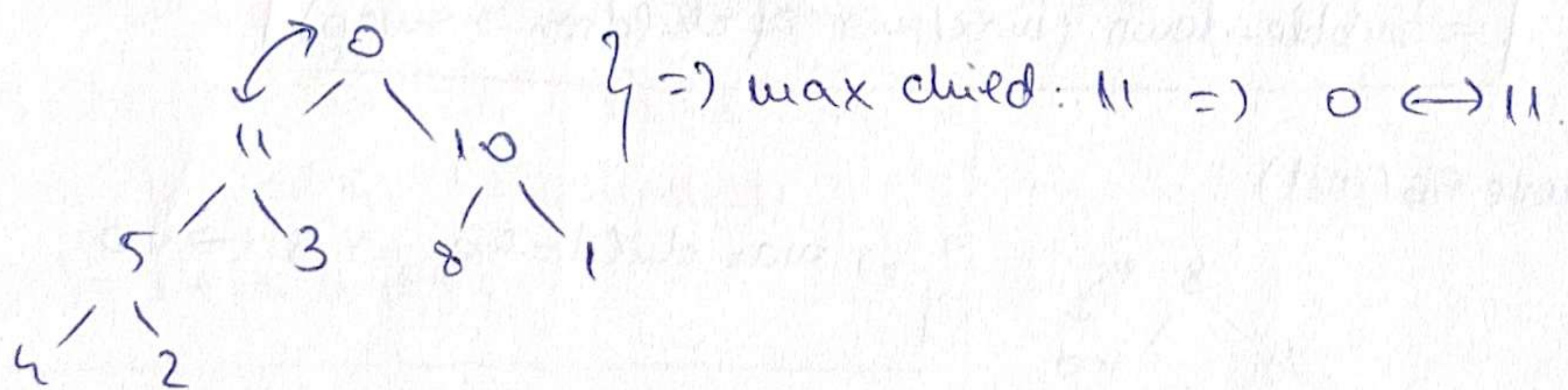


=>



(bubble down completed, valid ^{max} heap.)

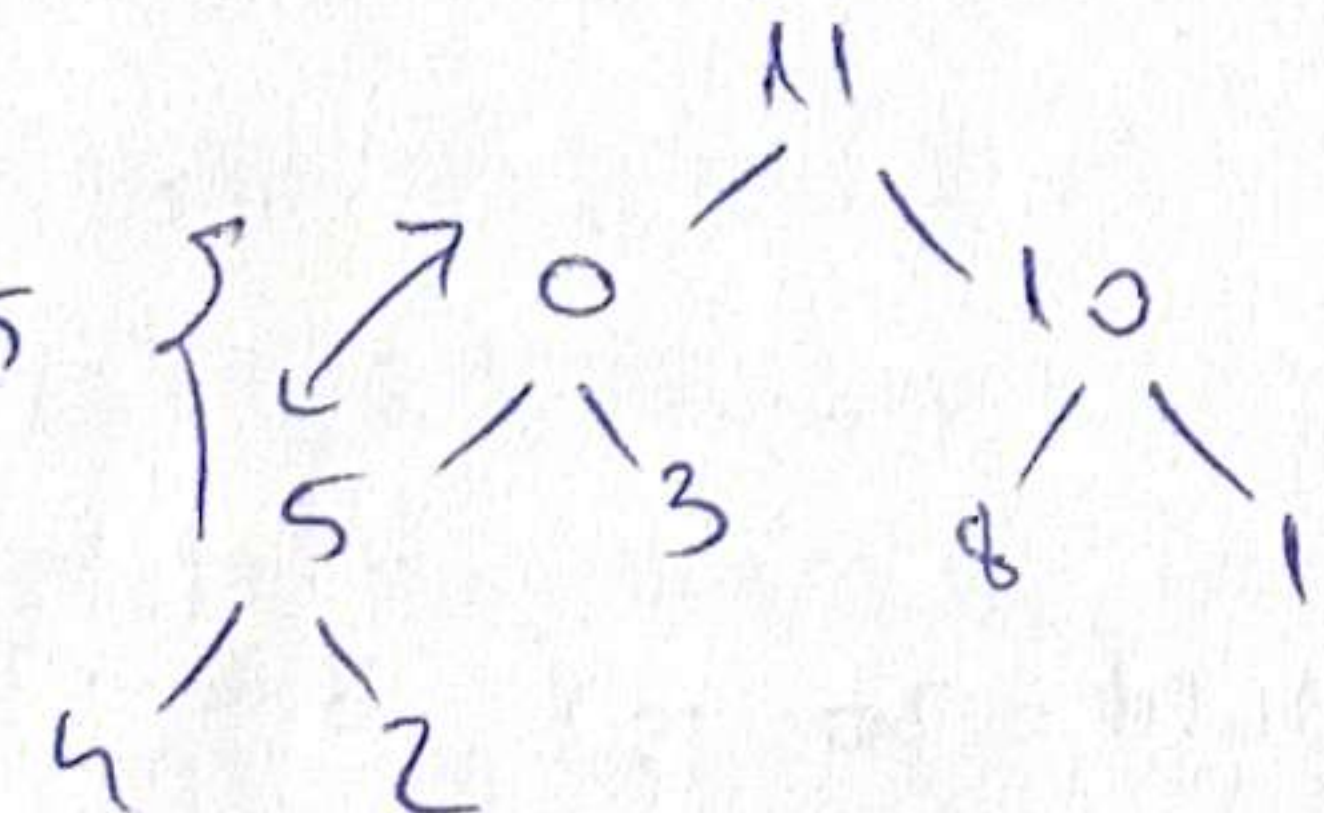
• Remove (10) root.



=>

max ch = 5

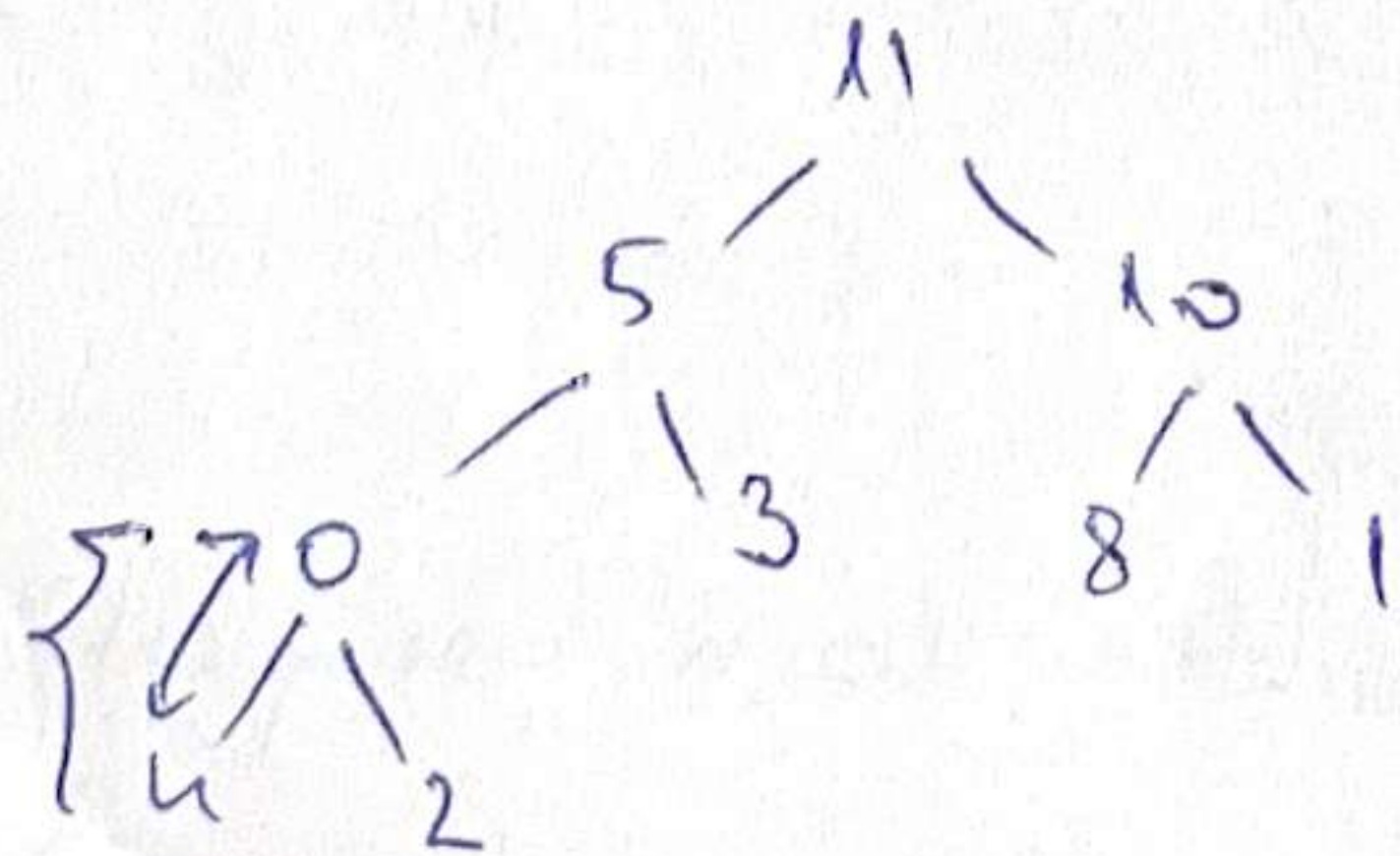
5 <-> 0



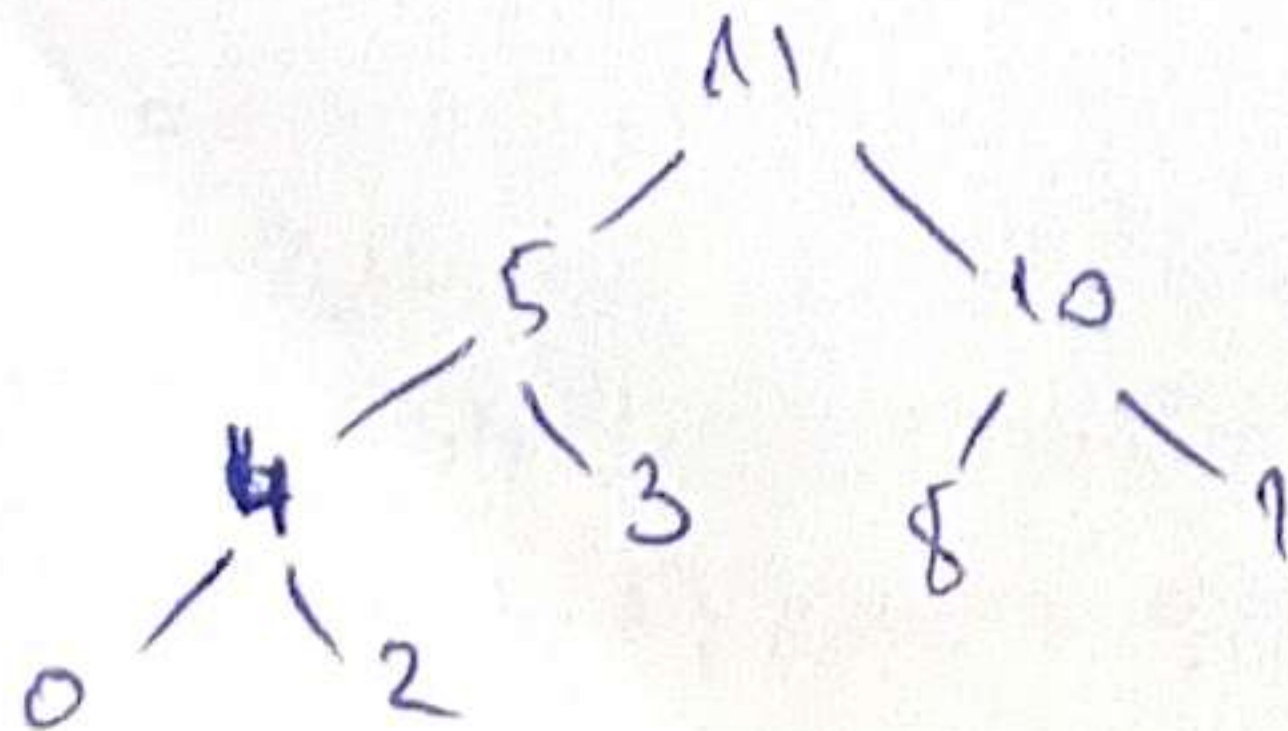
=>

max ch = 4

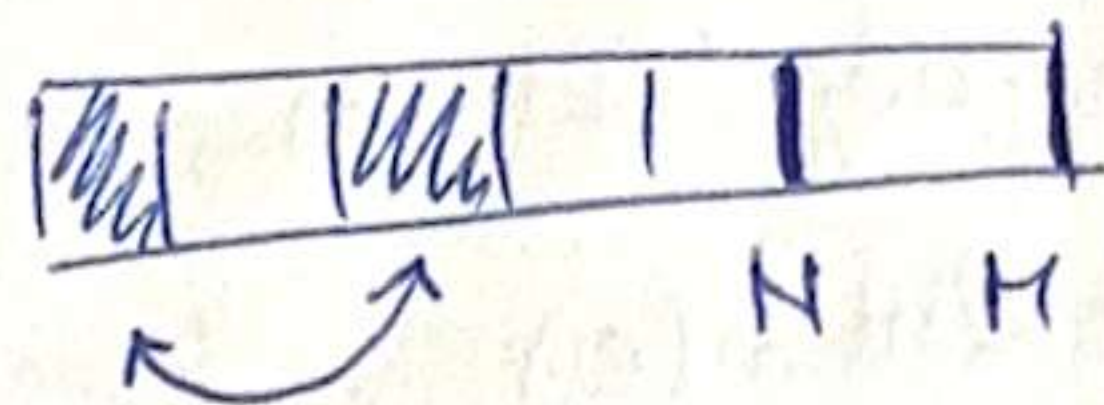
=> 0 <-> 4



=>



- 3) a) swap first elem. with the middle elem. in an array of size N containing N elems. ($N > 1$)



→ tightest bound for wc. (for op.)

Array access by index : $A[i]$ $\Rightarrow$ direct access, no traversal needed.

$temp = A[0]$
 $A[0] = A[N/2]$
 $A[N/2] = temp$

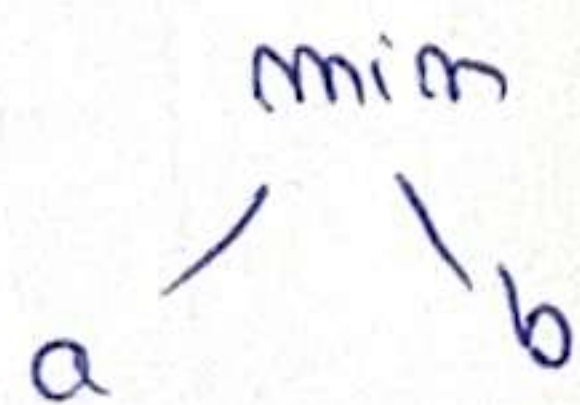
$\Rightarrow \Theta(1)$.

- b) pop elem. from queue represented on circular array with N^2 pos. (queue contains N elems.)

recy! Circular queue: front / rear
pop: $(front + 1) \% capacity$ $\Rightarrow \Theta(1)$.

- c) find second min. elem. in a Min-heap of N elems.

Min. heap $\Rightarrow$ root = min. $\Rightarrow$ second min. must be one of the root's children. $\Rightarrow \min(a, b) = \text{second min.}$



$\Rightarrow$

$\Rightarrow \Theta(1)$

- 3) Linked HT DS, optimised for getting elem. in sorted order.

Operations: search, add, remove.

Iterator: get elems in sorted order

- a) Representation: container, iterator.

Node:

info: Tkey

nextH: $\uparrow$ Node

nextL: $\uparrow$ Node

prevL: $\uparrow$ Node

Separate chaining

+

Sorted DLL

Linked HT:

m: Integer

T: ($\uparrow$ Node)[]

h: TFunction

head: $\uparrow$ Node

tail: $\uparrow$ Node

rel: Relation

size: Integer

HT $\rightarrow$ fast search, add, remove

LL $\rightarrow$ iteration order

nextH $\rightarrow$ for hash chains

nextL, prevL $\rightarrow$ for linked list order

6) ADD operation $\rightarrow$ specify + implement + complexity

function
subalgorithm add(lht, e):

pos $\leftarrow$ lht.h(e)

p $\leftarrow$ lht.T[pos]

while p $\neq$ NIL and [p].info $\neq$ e execute

└ p $\leftarrow$ [p].nextH

if p $\neq$ NIL then

add $\leftarrow$ false

else

allocate (newNode)

[newNode].info $\leftarrow$ e

[newNode].nextH $\leftarrow$ lht.T[pos]

lht.T[pos] $\leftarrow$ newNode

prev $\leftarrow$ NIL

current $\leftarrow$ lht.head

while current $\neq$ NIL and [current].info $<$ e execute

└ prev $\leftarrow$ current

└ current $\leftarrow$ [current].nextL

[newNode].prevL $\leftarrow$ prev

[newNode].nextL $\leftarrow$ current

if prev = NIL then

└ lht.head $\leftarrow$ newNode

else

└ [prev].nextL $\leftarrow$ newNode

if current = NIL then

└ lht.tail $\leftarrow$ newNode

else

└ [current].prevL $\leftarrow$ newNode

add $\leftarrow$ true

return add

? Complexity:

) Search + remove ; bc, wc, total.

→ Search: compute the hash position ($pos \leftarrow h(e)$), go to that position ($T[pos]$) and follow nextH until elem found NIL.

BC: $\theta(1) \rightarrow$ elem. is first in the chain.

WC: $\Theta(m) \rightarrow$ all elems. are in the same chain

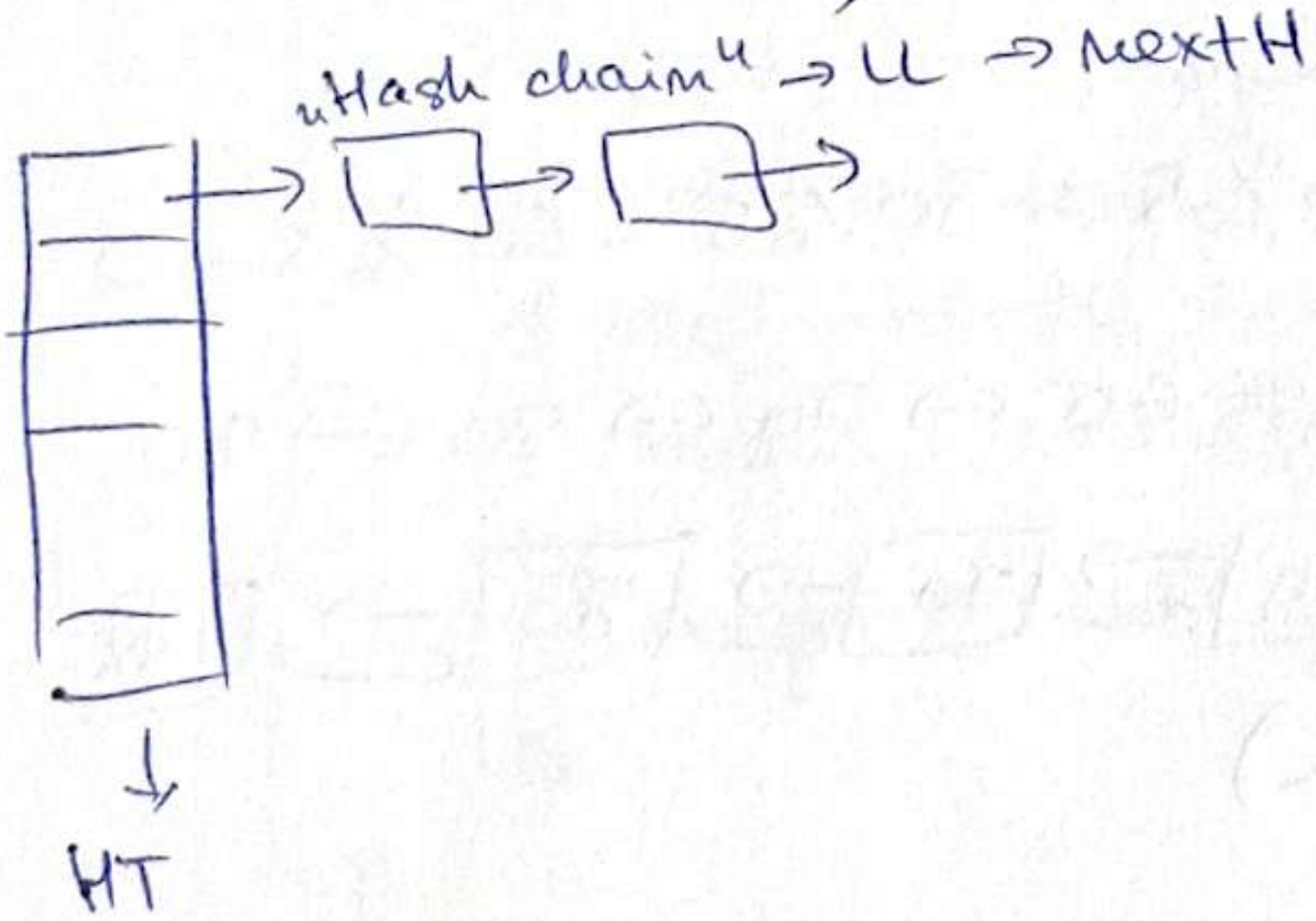
$$\Theta(1) \neq \Theta(n) \Rightarrow \text{Total: } O(n).$$

- remove: search + remove it from the hash chain (nextH links) and remove it from the sorted DL using prevL, nextL links.

BC: search $\Theta(1)$; remove - from hash chain: $\Theta(1)$ $\Rightarrow \Theta(1)$
 (in hash chain) - from DLL: $\Theta(1)$

WC: search $\Theta(m)$; remove - $\Theta(1)$ $\Rightarrow \Theta(m)$
(in hash chain) - $\Theta(1)$

$\Rightarrow$ Total: $O(n)$



 $\rightarrow$ Sorted DLL (all elems $\rightarrow$ next L, prev L).

d) iterator ; Bc, wc, total.

Iterative works with the sorted DLL

Representation:

lht: Linked HT

current : ↑ Node

subalgorithm first (it) is

subalgorithm first(it) is
 it.current $\leftarrow$ it.lht.head

$$Bc = Wc = \text{Total} = \Theta(1)$$

subalgorithm valid (it) is

subalgorithm valid(it) is

```
1 valid ← it.current ≠ Nil
```

$$B_c = W_c = \text{total} = \Theta(1)$$

Subalgorithm
function ~~getCurrent(it)~~ is ← not valid(it)
 if ~~it.current = Nil~~ then
 @throw exception
 getCurrent ← [it.current].info

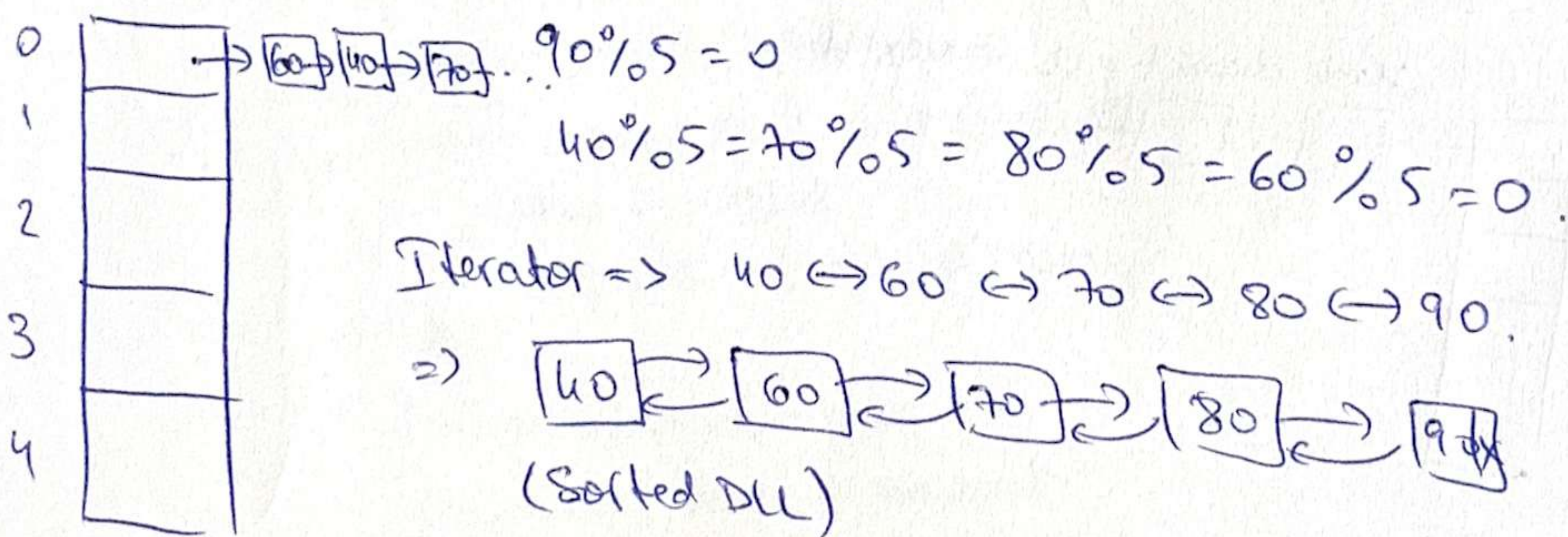
$$BC = WC = total = \Theta(1)$$

Subalgorithm next(it) is ← not valid(it)
 if ~~it.current = Nil~~ then
 @throw exception
 it.current ← [it.current].next

$$BC = WC = total = \Theta(1)$$

Whole iteration: $\Theta(n)$ because visits all elements ~~(next is called)~~

e) DS: 90, 40, 70, 80, 60 . $m = 5$



④ Binary Tree density (nr. elems / height) with one single traversal
 a) representation (linked w D.A)

BTNode:

info: TElem

left: TBTNode

right: TBTNode

BT:

root: TBTNode

b) density implementation

function density(tree) is

if tree.root = NIL then
 @throw exception

 nrNodes $\leftarrow$ 0

 height $\leftarrow$ 0

 compute(tree.root, nrNodes, height)

 density $\leftarrow$ nrNodes / height

subalgorithm compute(node, nrNodes, height) is

if node = NIL then

 nrNodes $\leftarrow$ 0

 height $\leftarrow$ -1

else

 compute([node].left, nrLeft, hLeft)

 compute([node].right, nrRight, hRight)

 nrNodes $\leftarrow$ 1 + nrLeft + nrRight

 if hLeft > hRight then

 height $\leftarrow$ 1 + hLeft

 else

 height $\leftarrow$ 1 + hRight

c) $BC = \Theta(n)$ $\quad \quad \quad \Rightarrow$ Total: $\Theta(n)$
 $WC = \Theta(n)$

BC = WC because the operation must visit every node to compute both the nr. of nodes and the height.

The solution did not use any auxiliary ADT's